

Clase 236 — Secure SDLC y filosofía shift-left

Parte: **11 — DevSecOps y seguridad del SDLC** · Fuente: *Agile Application Security* (Bell, Brunton-Spall, Smith, Bird) y NIST SP 800-218 (SSDF) 🕒 Duración estimada: **90 min** · Nivel: **Fundamentos**

Objetivo

Entender qué es un ciclo de vida de desarrollo de software seguro (Secure SDLC), por qué mover la seguridad "hacia la izquierda" del ciclo reduce drásticamente el coste y el riesgo, y qué controles concretos corresponden a cada fase. Al terminar, el alumno sabrá dibujar el SDLC de su organización y ubicar en él las prácticas y herramientas de esta parte.

Resultados de aprendizaje

Al finalizar, el alumno podrá:

- 1. **Explicar** la diferencia entre seguridad "bolt-on" (al final) y "built-in" (integrada), con datos de coste de remediación.
- 2. **Mapear** los controles de seguridad a cada fase del SDLC (requisitos, diseño, código, build, test, despliegue, operación).
- 3. **Justificar** shift-left frente a stakeholders usando métricas (tiempo de detección, coste de corrección).
- 4. **Seleccionar** un modelo de madurez (OWASP SAMM o BSIMM) para evaluar el estado actual.
- 5. **Diseñar** un flujo mínimo viable de DevSecOps para un equipo pequeño.

Temas

#	Tema	Por qué importa
1	Fases del SDLC	Es el mapa donde ubicaremos todos los controles
2	Coste de remediación por fase	Justifica económicamente el shift-left
3	Shift-left vs shift-right	Complementarios: prevenir y también observar en producción
4	Controles por fase (gates)	Define qué se automatiza y dónde
5	OWASP SAMM / BSIMM	Modelos de madurez para medir progreso
6	NIST SSDF (800-218)	Marco de referencia reconocido y auditable
7	Roles y responsabilidad compartida	"Security is everyone's job", no solo de AppSec

Definiciones y características

- **Secure SDLC**: proceso de desarrollo donde las actividades de seguridad están integradas en cada fase, no añadidas al final. *Característica*: los controles son parte del "definition of done".
- **Shift-left**: mover las actividades de seguridad lo más temprano posible en el ciclo. *Característica*: cuanto antes se detecta un defecto, más barato es corregirlo.

- **Shift-right:** complementar con observabilidad y pruebas en producción (feature flags, canary, RASP). *Característica:* asume que no todo se puede prever antes del despliegue.
- **Security gate:** punto del pipeline donde una comprobación puede bloquear el avance. *Característica:* automático y objetivo, no una revisión manual subjetiva.
- **OWASP SAMM:** modelo de madurez con 5 funciones de negocio y prácticas medibles. *Característica:* prescriptivo y agnóstico de tecnología.
- **BSIMM:** modelo descriptivo basado en observar qué hacen organizaciones reales. *Característica:* benchmarking, no prescripción.

Herramientas y preparación

Esta clase es conceptual/estratégica; el "laboratorio" es de diseño. Necesitarás:

- Una pizarra digital (Excalidraw, Miro) o papel para diagramar el SDLC.
- La hoja de auto-evaluación de **OWASP SAMM v2** (descargable como toolbox en Excel desde [owasp samm.org](https://owasp.org/www-project-samm/)).
- Acceso al documento **NIST SP 800-218** para consultar las prácticas PO/PS/PW/RV.
- Un repositorio de ejemplo propio para ubicar dónde encajaría cada control.

Laboratorio guiado

Ejercicio aplicado de diseño y evaluación (no ofensivo):

1. **Dibuja tu SDLC actual.** Lista las fases reales por las que pasa un cambio en tu equipo: idea → requisitos → diseño → codificación → PR/review → build → test → despliegue → operación.
2. **Ubica los controles existentes.** Marca en cada fase qué comprobación de seguridad ya existe hoy (aunque sea "ninguna"). Sé honesto.
3. **Identifica los huecos.** Para cada fase sin control, anota una práctica candidata de esta parte (p. ej. threat modeling en diseño, SAST en PR, SCA en build).
4. **Descarga el toolbox de OWASP SAMM v2** y completa la auto-evaluación de al menos la práctica "Secure Build" y "Security Testing". Anota tu nivel actual (0–3).
5. **Estima el coste.** Toma un bug de seguridad real que tu equipo haya arreglado en producción y estima qué habría costado detectarlo en la fase de código (revisa el modelo de IBM Systems Sciences Institute: ~1x en diseño, ~15x en test, ~100x en producción).
6. **Prioriza 3 gates.** Elige los tres controles automatizados de mayor impacto/menor esfuerzo para introducir primero y justifícalos.

Ejercicios

1. Enumera las fases del SDLC y asigna a cada una un control de seguridad concreto.
2. Explica con un ejemplo por qué un fallo de diseño es más caro que un fallo de código.
3. Diferencia shift-left de shift-right con un caso donde ambos son necesarios.
4. Compara OWASP SAMM y BSIMM: cuándo usarías cada uno.
5. Redacta un "definition of done" que incluya al menos tres criterios de seguridad.
6. Diseña un roadmap de 90 días para llevar una práctica de SAMM de nivel 0 a nivel 1.

Reto verificable

Produce un documento de **una página** con el mapa del SDLC de un proyecto real (o de ejemplo), con al menos un control automatizado por fase y una auto-evaluación SAMM de tres prácticas.

Criterio de aceptación: el documento (a) cubre todas las fases desde requisitos hasta operación, (b) asigna al menos un gate automatizable a cada fase, (c) incluye puntuación SAMM 0–3 de tres prácticas con justificación, y (d) prioriza 3 mejoras con criterio impacto/esfuerzo explícito.

Errores comunes

Síntoma / mensaje	Causa y cómo arreglar
"Shift-left = comprar herramientas SAST"	Confundir herramienta con proceso. Empieza por el proceso y los gates, la herramienta viene después.
El pipeline se llena de falsos positivos y se ignora	No se afinó ni se definió qué rompe el build. Calibra severidades y empieza en modo "warning".
AppSec sigue siendo cuello de botella	No se delegó ownership al equipo de desarrollo. Forma champions (clase 248) y automatiza.
Se mide "número de vulnerabilidades" y sube	Métrica sin contexto de riesgo. Mide tiempo de remediación y densidad por exposición.
Todo el foco en shift-left, nada en producción	Falta observabilidad. Complementa con shift-right (logging, detección en runtime).

Preguntas frecuentes

? ¿Shift-left significa que los desarrolladores hacen todo el trabajo de seguridad? No. Significa que las herramientas y prácticas de seguridad están disponibles para ellos temprano y de forma automatizada, con AppSec como habilitador y guardián de estándares, no como ejecutor único.

? ¿Necesito madurar en SAMM antes de automatizar? No es secuencial estricto. SAMM te da un diagnóstico; puedes empezar a automatizar los gates de mayor valor mientras subes de nivel en paralelo.

? ¿Secure SDLC aplica a metodologías ágiles? Sí, y es donde más brilla. En ágil los controles se integran en cada iteración y en el pipeline, no en un "gate de seguridad" al final del proyecto.

? ¿SSDF y SAMM compiten? No. SSDF (NIST) es un marco de prácticas de referencia; SAMM es un modelo de madurez para medirte. Se complementan: SSDF dice "qué", SAMM dice "cuánto de maduro".

Referencias

- NIST SP 800-218 SSDF — <https://csrc.nist.gov/pubs/sp/800/218/final>
- OWASP SAMM v2 — <https://owasp samm.org/>
- BSIMM — <https://www.bsimm.com/>
- Bell, Brunton-Spall, Smith, Bird, *Agile Application Security*, O'Reilly 2017.
- OWASP DevSecOps Guideline — <https://owasp.org/www-project-devsecops-guideline/>

Siguiente clase

Clase 237 - Modelado de amenazas: STRIDE y DREAD